Mathematical Reconstruction Report

Space Plugin and Scripted Falcon Guidance

A code-grounded explanation of the formulas, conversions, controllers, approximations, and trajectory equations implemented in the PowerEngine Space sandbox.

To
Elon Musk

From
Lieutenant Lopez

Date
5 April 2026

Primary Sources
Sandbox/Space/plugins/space/*.go
Sandbox/Space/plugins/space/scripts/*.as
Sandbox/Space/Port/physics/*.py

The scope of this report is the math that is actually implemented, not ideal textbook behavior. Where the Space stack uses approximations, display-only curves, or kinematic shortcuts, those are called out directly.

Profile Handling Note
The Falcon mission profile in Sandbox/Space/profiles/falcon_Earth_to_Mars_1.tsv was scanned as a consistency witness for logged fields and phase behavior, but the profile itself is intentionally not reproduced in this document.

1. Scope and Method

This report covers the mathematical surfaces in the Go plugin, the AngelScript control scripts, and the mirrored Python reference helpers that the plugin explicitly cites. The analysis follows the implementation path from raw orbital elements, through body-state construction, through the runtime N-body simulation, and finally into the Falcon guidance logic, telemetry generation, and visualization math.

The most important implementation files are:

- physics.go: constants, vector algebra, orbital-period helper, HSV conversion, and the momentum integrator.
- simulation.go: pairwise Newtonian forces, Sun-only asteroid forcing, and simulation substepping.
- solar_system.go: J2000 element evaluation, Kepler solving, orbital state-vector construction, asteroid mass synthesis, and initial heliocentric states.
- objects.go: AU and km/s conversion into plugin units, body momentum initialization, and logarithmic visual scaling.
- SolarSystem.as: Falcon ascent, transfer planning, transfer interpolation, approach and landing control, telemetry derivation, camera math, and HUD formulas.
- OrbitMath.as and SolarLighting.as: compact geometry and visual-scaling helpers.
- Port/physics/classical.py and Port/physics/integrator.py: reference formulas mirrored by the plugin.

> One governing principle shaped this analysis: several quantities with names like "burn," "capture," or "intercept" are not fully dynamic rocket-propagation quantities. In more than one phase, the code computes them for display, planning, or bookkeeping while the actual state advance is performed by a separate analytic or kinematic path. That distinction is documented each time it occurs.

2. Units, Frames, and Coordinate Conventions

The Space stack mixes three distinct numerical frames. Understanding the conversion contract is the key to reading every other formula correctly.

Quantity	Implemented Value	Meaning	Primary Use
AUInMeters	1.496e11	1 astronomical unit in meters	SI-to-orbit conversion
GameUnitsPerAU	1000.0	Plugin orbit-space scale	Heliocentric body simulation
BaseDistance	GameUnitsPerAU / AUInMeters	Meters to orbit game-units	Velocity conversion and forces
UnitConversion	AUInMeters / GameUnitsPerAU	Orbit game-units to meters	Distance and force conversion
F9_METERS_PER_GU	5.0	Local launch/landing scale	Falcon local-frame ascent
ORBIT_GU_PER_METER	1000 / AUInMeters	Orbit-frame meters to game-units	World-frame Falcon guidance

```
Base distance scale
BaseDistance = GameUnitsPerAU / AUInMeters
```

```
Inverse conversion
UnitConversion = AUInMeters / GameUnitsPerAU
```

```
Local Falcon conversion
F9_GU_PER_METER = 1 / F9_METERS_PER_GU
```

```
Orbit-frame metric conversion
ORBIT_GU_PER_METER = 1000 / AUInMeters
```

The Go plugin simulates heliocentric body positions directly in orbit game-units. The script then remaps the returned axes for rendering so that the engine can use XZ as the horizontal plane and Y

as vertical.

```
Script-side coordinate remap
(xrender, yrender, zrender) = (xplugin, zplugin, yplugin)
```

This is why some script formulas appear to use "Y" as height even though the Go layer stores heliocentric vectors as ordinary XYZ tuples.

3. Fundamental Constants and Physical Parameters

The plugin math is classical and explicitly Newtonian. The physically active constants are the gravitational constant GReal, the Solar standard gravitational parameter through SunMassKg, the Mars gravitational parameter for Falcon world guidance, and the Falcon performance constants used in ascent and landing calculations.

Symbol	Implementation	Interpretation
G	6.67430e-11 m3 kg-1 s-2	Newtonian gravitational constant
Msun	1.989e30 kg	Used for heliocentric period and speed formulas
musun	SUN_MU = 1.3271244e20 m3/s2	Used in Falcon transfer and transfer HUD formulas
muMars	MARS_MU = 4.282837e13 m3/s2	Used in world-frame approach gravity compensation
g0	9.80665 m/s2	Used for Falcon thrust-to-mass and mass flow
Tmax	7,607,000 N	Falcon maximum thrust

```
Screen-space gravitational parameters used by scripts
muscreen = muSI * ORBIT_GU_PER_METER3
```

The scripts convert SI gravitational parameters into orbit-game-unit space by multiplying by the cube of the orbit-frame meter scale. That is exactly what is required because gravitational acceleration has dimensions of distance cubed divided by time squared over radius cubed.

4. Solar-System State Construction from Orbital Elements

The Solar System bodies are seeded from J2000 Keplerian elements with linear drift rates per Julian century. The plugin evaluates those elements at a fixed epoch near 2026-04-03 and then converts them into heliocentric state vectors.

Governing implementation: Sandbox/Space/plugins/space/solar_system.go

4.1 Element Evaluation at Epoch

For each body, the time-adjusted elements are evaluated as affine functions of the Julian-century parameter T.

```
a = a0 + a1 T
e = e0 + e1 T
i = i0 + i1 T
Omega = Omega0 + Omega1 T
varpi = varpi0 + varpi1 T
L = L0 + L1 T
```

4.2 Mean Anomaly and Argument of Perihelion

```
M = L - varpi
omega = varpi - Omega
```

Each angle is then wrapped into the interval [0, 360) by the helper mod360 before converting to radians.

4.3 Kepler Equation Solver

The eccentric anomaly E is obtained with Newton-Raphson iteration applied to the classical Kepler equation M = E - e sin(E).

```
En+1 = En + dE
dE = (M - (E - e sin(E))) / (1 - e cos(E))
```

The implementation caps the loop at 50 iterations and exits early when |dE| < 1e-12.

4.4 True Anomaly

```
nu = 2 atan2(sqrt(1 + e) sin(E/2), sqrt(1 - e) cos(E/2))
```

4.5 Orbital-Plane Radius, Position, and Velocity

The state-vector builder computes the heliocentric radius in astronomical units and the orbital-plane position and velocity before rotating them into the ecliptic frame.

```
r = a (1 - e2) / (1 + e cos(nu))
```

```
Orbital-plane position
xo = r cos(nu)
yo = r sin(nu)
```

```
Specific angular momentum path
p = a AUInMeters (1 - e2)
h = sqrt(mu p)
```

```
Orbital-plane velocity components
vx,o = -(mu / h) sin(nu)
vy,o = (mu / h) (e + cos(nu))
```

```
Unit conversion
vkm/s = vm/s / 1000
```

4.6 Rotation into Ecliptic Coordinates

The code constructs the P and Q column vectors for the standard orbital-plane-to-ecliptic rotation parameterized by omega, Omega, and i.

```
Px = cos(omega) cos(Omega) - sin(omega) sin(Omega) cos(i)
Py = cos(omega) sin(Omega) + sin(omega) cos(Omega) cos(i)
Pz = sin(omega) sin(i)
```

```
Qx = -sin(omega) cos(Omega) - cos(omega) sin(Omega) cos(i)
Qy = -sin(omega) sin(Omega) + cos(omega) cos(Omega) cos(i)
Qz = cos(omega) sin(i)
```

```
pos = xo P + yo Q
vel = vx,o P + vy,o Q
```

4.7 Whole-System Drift

After the heliocentric velocity is constructed, the plugin adds a common drift vector to all bodies. This is not an orbital-mechanics correction. It is a deliberate visual framing decision so the whole system traces a spiral-like path in world space when viewed in the right camera mode.

```
vfinal = vorbital + vdrift
```

5. Asteroid Cloud Formulas

Asteroids are not loaded as full perturbed ephemerides. They are generated from near-Earth-asteroid records with a deliberately simplified Source-style path: size from absolute magnitude, mass from spherical volume and rocky density, and initial motion from circularized solar orbit assumptions at a seeded phase.

Governing implementation: loadNEAAsteroidStates in solar_system.go

5.1 Diameter and Radius from Absolute Magnitude

```
diameterkm = 103.123 - H / 5
radiuskm = diameterkm / 2
```

5.2 Mass from Volume and Density

```
mass = (4 / 3) pi (radiusm)3 rho
with rho = 2500 kg/m3
```

5.3 Initial Orbital Estimate

The asteroid seed uses the entry perihelion distance as the working orbital radius, enforces zero eccentricity, and assigns a deterministic phase using a hash-derived fraction of 2pi. The body period is then estimated with the circular-orbit scaling law.

```
phase = hash(name) * 2pi
perioddays = 365.25 sqrt(rAU3)
```

> This is intentionally lighter than a full small-body propagator. The asteroid cloud is mathematically coherent, but it is simplified for rendering and interaction cost rather than for long-horizon ephemeris fidelity.

6. Body-State Conversion, Force Law, and Integrator

Once the initial heliocentric states are assembled, the runtime simulation advances them in orbit game-units using Newtonian pairwise gravity and a symplectic-Euler-like momentum step.

Governing implementation: objects.go, physics.go, and simulation.go

6.1 Input-State Conversion

```
Position
xGU = xAU * GameUnitsPerAU
```

```
Velocity
vGU/s = vkm/s * 1000 * BaseDistance
```

```
p = m v
```

6.2 Newtonian Force in Orbit-Game Units

The runtime force law starts in SI Newtons and then divides by the distance conversion so the stored force has units of kg * GU / s2.

```
FN = G m1 m2 / rm2
```

```
rm = rGU * UnitConversion
FGU = FN / UnitConversion
```

```
Fvector = FGU * (r / |r|)
```

Primary bodies interact with one another pairwise. Asteroid-cloud bodies are then forced only by the Sun, which is a deliberate cost-saving and aesthetic simplification.

6.3 Symplectic Momentum Integrator

The implementation and its mirrored Python reference both document the same three-line update rule:

```
pnew = p + F dt timeScale
vnew = pnew / m
xnew = x + vnew dt timeScale
```

This is described in comments as "leapfrog / symplectic Euler" and it has the key structural feature that momentum is updated before the position update uses the new velocity.

6.4 Substep Strategy

To avoid extremely large integration jumps under high time-scale settings, the simulation slices one frame into substeps when the simulated elapsed time exceeds one hour.

```
rawStep = |dt * timeScale|
subCount = ceil(rawStep / 3600) capped at 24
subDt = dt / subCount
```

7. Classical Mechanics Helpers and Color Mapping

The plugin mirrors a compact classical-mechanics module from the Python port. These helpers are not responsible for the actual N-body advance, but they drive diagnostics and orbit-color presentation.

7.1 Orbital Period

```
T = 2 pi sqrt(r3 / (G M))
```

The plugin first converts the orbit-frame distance to meters, then evaluates the above expression relative to the Sun. This is used as a period estimate for the currently simulated heliocentric radius.

7.2 Circular-Orbit Speed

```
v = sqrt(G (Mcentral + Mbody) / r)
```

7.3 Period-to-Hue Mapping

```
t = clamp(periodDays / (10 * 365), 0, 1)
hue = 240 (1 - t)
```

The hue is then converted to RGB by the piecewise sector-based HSV conversion in HsvToRgb. This is purely visual math, but it is still part of the implemented formula stack.

8. Script-Side Geometry and Predicted Body Motion

SolarSystem.as builds several local geometric helpers on top of the simulated body positions.
These functions are used throughout Falcon guidance and camera placement.

8.1 Reference Geometry

```
OrbitXZ(angle, radius, height) = (radius cos(angle), height, radius sin(angle))
```

8.2 Body Velocity from Finite Difference

```
vbody = (x(t) - x(t - Deltat)) / Deltat
```

Script-side body velocity is not queried from the Go integrator directly. Instead, it is reconstructed from consecutive body positions using the current simulated time advance.

8.3 Predicted Body Position by Angular Rotation

Future body motion is approximated by rotating the current heliocentric relative vector around the engine's vertical axis using the stored orbital period, not by propagating the full N-body state forward.

```
advance = 2 pi daysAhead / periodDays
```

```
angularSign = rx vz - rz vx
```

```
rpred = RotateAroundY(r, advance * sign)
```

> This prediction is explicitly a circularized angular extrapolation. It ignores changing radius, perturbations, and any future evolution of the N-body state. The transfer planner therefore uses an approximation of the target body's future heliocentric location, not a numerically propagated intercept solution.

8.4 Local Body Basis

```
Up
up = normalize(bodyPos - sunPos)
```

```
Forward
tangential = v - up (v dot up)
forward = normalize(tangential)
```

```
Right
right = normalize(forward cross up)
```

```
Rotate around Y
x' = x cos(a) - z sin(a)
z' = x sin(a) + z cos(a)
```

9. Falcon Local-Frame Ascent Math

The Falcon booster uses two regimes. First it ascends in a local ground-relative frame measured in the Falcon's own 5-meters-per-unit scale. Later it switches to orbit-world coordinates for the interplanetary route.

9.1 Booster Mass

```
m = mdry + max(mprop - mfuelSpent, 0)
```

9.2 Local Gravity and Drag

```
Local gravity
ag = (0, -g0 * F9_GU_PER_METER, 0)
```

```
Implemented drag
ad = -k |v| v
with k = 0.0025
```

The drag term is quadratic in speed because the velocity vector is multiplied by its magnitude.

9.3 Maximum Thrust Acceleration

```
amax = (Tmax / m) * F9_GU_PER_METER
```

9.4 Injection Direction and Tower Alignment

The ascent controller tries to align the local horizontal motion with the heliocentric prograde direction of the launch planet. Lateral error is simply the horizontal component of the injection vector.

```
Lateral error
elat = (dirx, 0, dirz)
```

```
Lateral velocity feedback
vlat,fb = (-vx, 0, -vz)
```

```
align = clamp(1 - |elat| / 12, 0, 1) * 100
```

9.5 Grid Fin Commands

```
finCmdX = clamp(0.10 ex + 0.30 vx,fb, -1, 1)
finCmdZ = clamp(0.10 ez + 0.30 vz,fb, -1, 1)
```

```
yawgridfin = finCmdX * 30 deg
pitchgridfin = -finCmdZ * 30 deg
```

9.6 Gravity Turn

The ascent thrust vector starts nearly vertical and progressively leans into the heliocentric injection direction as altitude increases.

```
gravityTurn = clamp(altitudeMeters / 1800, 0, 1)
```

```
thrustDir = normalize(
(dirx (0.15 + 0.95 gravityTurn),
1 - 0.72 gravityTurn,
dirz (0.15 + 0.95 gravityTurn)))
```

9.7 Mass Flow

```
m-dot = throttle * Tmax / (Isp g0)
```

9.8 Local State Update

```
a = ag + ad + throttle * amax thrustDir
v = v + a dt
x = x + v dt
```

The ascent phase exits when either the phase timer exceeds 18 seconds or the local altitude exceeds 2200 meters.

10. Falcon Transfer Planning

The interplanetary transfer planner is Hohmann-like in its scalar formulas but not a full Lambert solve. It uses the current heliocentric start vector, a circularized prediction of the target's future position, and a derived elliptical radius law to generate the cruise path.

10.1 Transfer Time

The nominal transfer time is computed from the semi-major axis in physical meters.

```
a = (r1 + r2) / 2
```

```
TOF = pi sqrt(a3 / musun)
```

10.2 Transfer Speeds and Injection Delta-v

```
Circular speed at departure
vcirc = sqrt(mu / r1)
```

```
Transfer speed at departure
vtrans = sqrt(mu (2 / r1 - 1 / a))
```

```
Deltav1 = max(vtrans - vcurrent, 0)
```

If the script-side estimate of current heliocentric departure speed is non-positive, the code falls back to the circular speed estimate. The burn direction is simply heliocentric prograde.

10.3 Stored Transfer Shape

```
Semi-major axis
a = 0.5 (r1 + r2)
```

```
Eccentricity
e = (r2 - r1) / (r1 + r2)
```

10.4 Logged Burn Acceleration

The script stores a nominal two-hour burn duration and converts the scalar injection delta-v into a constant acceleration vector for telemetry purposes.

```
burnTime = 7200 s
aburn,logged = dirburn * (Deltav1 / burnTime)
```

> This burn acceleration is not what actually propagates the transfer trajectory. During cruise, the code sets the Falcon world position and velocity directly from the analytic transfer curve. The burn vector is accumulated into displayed total delta-v and logged into the profile, but cruise state itself is not integrated from thrust.

10.5 Lead Angle

```
leadAngle = 360 * daysAhead / targetPeriodDays mod 360
```

11. Transfer Curve Interpolation and Cruise Kinematics

Cruise is driven by a parameterized trajectory indexed by normalized transfer progress p. The curve uses the classic ellipse radius formula, but the target angle is taken from the predicted target position rather than from a Lambert boundary-value solution.

11.1 Transfer Progress

```
p = clamp(elapsedDays / transferDays, 0, 1)
```

11.2 Angular Sweep

The code computes start and target heliocentric angles in the script's horizontal plane and then wraps the sweep direction according to the sign of the prograde cross-product.

```
anglestart = atan2(rstart,z, rstart,x)
angletarget = atan2(rtarget,z, rtarget,x)
```

```
progradeSign = rstart,x tz - rstart,z tx
```

11.3 Radius Law Along the Transfer

```
nu = pi p
```

```
r(p) = a (1 - e2) / (1 + e cos(nu))
```

The Y component is not solved orbitally. It is linearly interpolated between start and target relative heights.

```
y(p) = ystart + (ytarget - ystart) p
```

11.4 Parametric Cruise Position

```
relPos(p) = (cos(angle(p)) r(p), y(p), sin(angle(p)) r(p))
```

11.5 Cruise Velocity from Finite Difference

Instead of differentiating the transfer analytically, the code samples nearby progress points and divides by the corresponding physical time interval.

```
relVel(p) approximately [relPos(p + s) - relPos(p - s)] / [2 s transferDays 86400]
```

11.6 Actual Cruise State Update

```
worldPos = sunPos + relPos(p)
worldVel = sunVel + relVel(p)
```

That pair of assignments is the real cruise propagator in the current implementation.

11.7 Arc Length Estimate for HUD

The displayed traveled and remaining transfer distance uses the Ramanujan perimeter approximation for an ellipse, then halves it to obtain the nominal transfer arc.

```
b = a sqrt(1 - e2)
```

```
perimeter approximately pi [3(a + b) - sqrt((3a + b)(a + 3b))]
```

```
transferArc approximately 0.5 * perimeter
```

```
traveled = transferArc * p
remaining = max(transferArc - traveled, 0)
```

12. Mars Approach Targeting and Velocity Capture

The script contains a complete world-frame approach and landing controller relative to the target planet. Even though the current sampled Falcon route terminates directly in ARRIVED, the approach math is fully implemented and deserves separate treatment.

12.1 Mars Pad and Capture Geometry

The target pad is placed above the planet along the target body's current outward radial direction. The effective radius includes the physical body radius plus half the booster height and a small clearance offset.

```
pad = marsPos + upbody [Rmars + clearance]
```

12.2 Entry-Up to Pad-Up Blend

The approach target is not always directly above the pad. It blends between the entry-side up vector captured at transfer handoff and the pad's own up vector as altitude falls.

```
blend = clamp((45000 - altitudeMeters) / 33000, 0, 1)
```

```
targetUp = normalize(entryUp (1 - blend) + padUp blend)
```

```
approachTarget = marsPos + targetUp [surfaceRadius + 8000 m]
```

12.3 Velocity Capture at Approach Handoff

On transfer-to-approach capture, the script limits lateral and vertical relative velocities so the landing controller starts from bounded values.

```
vvertical = clamp(v dot up, -90 m/s, 60 m/s)
```

```
vlateral = clampLength(v - up (v dot up), 450 m/s)
```

```
vworld,new = vMars + vlateral + up vvertical
```

13. World-Frame Gravity and Powered Descent Controller

The Mars guidance loop is a relative-motion controller that uses Sun and Mars gravity, decomposes relative motion into lateral and vertical components, and applies a PD-style acceleration command with thrust saturation.

13.1 Point-Mass Gravity in Orbit Game-Units

```
delta = attractorPos - pos
r2 = delta dot delta
invR = 1 / sqrt(r2)
ag = delta * (mu invR3)
```

This is the standard mu r / |r|^3 acceleration form, evaluated separately for the rocket relative to the Sun, Mars relative to the Sun, and the rocket relative to Mars.

13.2 Relative Motion Decomposition

```
relPos = rocketPos - targetPos
relVel = rocketVel - targetPlanetVel
```

```
altitude = relPos dot up
lateralPos = relPos - up altitude
```

```
vvertical = relVel dot up
vlateral = relVel - up vvertical
```

13.3 Controller Gains

The controller uses one gain set for the broader Mars approach and a stronger gain set for the landing phase.

Phase	Lateral Kp	Lateral Kd	Vertical Kp	Vertical Kd	Thrust Cap
Mars approach	0.12	0.42	0.22	0.55	0.75 * maxWorldAccel
Landing	0.22	0.60	0.45	0.95	maxWorldAccel

13.4 Desired Relative Acceleration

```
adesired = -Kp,lat lateralPos - Kd,lat lateralVel
```

```
altitudeError = altitude - targetAltitude
```

```
adesired = adesired + up (-Kp,vert altitudeError - Kd,vert vvertical)
```

13.5 Relative Gravity Compensation

```
arelGravity = (asun on rocket - asun on Mars) + aMars on rocket
```

```
acmd = clampLength(adesired - arelGravity, athrust,max)
```

```
aworld,total = asun on rocket + aMars on rocket + acmd
```

This is the mathematically central landing equation in the script. It makes the controller operate in relative coordinates while still respecting the world-frame gravitational field.

13.6 World State Update

```
v = v + a dt
x = x + v dt
```

13.7 Surface Clamp

If the propagated state passes below the pad plane, the script projects the position back onto the surface and removes any inward component of relative velocity.

```
if altitudePost < 0, then x = pad + (relPost - up altitudePost)
```

```
inward = (v - vMars) dot up
if inward < 0, then v = v - up inward
```

13.8 Phase Transition Thresholds

```
approach to landing if altitude <= 12000 m and lateral error <= 5000 m
```

```
landing to arrived if altitude <= 25 m and |vvertical| <= 2 m/s and lateral speed <= 1.2 m/s
```

14. Telemetry, Logged Observables, and Profile-Derived Math

The Falcon profile logger computes several derived observables from the simulated state. These formulas matter because the profile is the public-facing record of how the code interprets the flight.

14.1 Heading and Pitch

```
speedhorizontal = sqrt(vx2 + vz2)
```

```
heading = normalizeDegrees(atan2(vx, vz) * 180 / pi)
```

```
pitch = atan2(vy, speedhorizontal) * 180 / pi
```

14.2 Heliocentric Longitude and Latitude

```
lon = normalizeDegrees(atan2(rz, rx) * 180 / pi)
```

```
lat = atan2(ry, sqrt(rx2 + rz2)) * 180 / pi
```

14.3 Range and Speed Conversions

```
originRangeAU = |worldPos - originPos| / 1000
```

```
speedm/s = orbitGuToMeters(|relVel|)
```

```
padRangekm = orbitGuToMeters(|pad - worldPos|) / 1000
```

14.4 Display-Reported Delta-v

The script accumulates the magnitude of the commanded acceleration over time and labels the resulting scalar as delta-v. Mathematically that is correct if interpreted as the time integral of acceleration magnitude.

```
dvaccum = integral |acmd| dt
```

14.5 Phase-Dependent Sample Rate

Phase	Sample Interval
Ascent	0.25 s
Transfer	1800 s
Target approach	60 s
Target landing	2 s
Arrived	0 s

The scanned Falcon profile confirmed the presence of these telemetry channels and their phase-driven sampling behavior. No raw profile rows are reproduced here.

15. Secondary Visualization Math

The Space sandbox also contains mathematically nontrivial display logic. It is not propulsion or orbital dynamics, but it still shapes how the simulation is interpreted visually.

15.1 Body Visual Scale

```
scale = clamp(0.6 log10(max(radiusKm, 1)), 0.3, 5.0)
```

15.2 Sun Glow Scaling

```
glowScale = sunBaseScale * (1 + 0.5 * sunIntensity)
```

15.3 Falcon Display Smoothing

```
blend = clamp(14 dt, 0, 1)
```

```
displayPos = displayPos + (truePos - displayPos) blend
```

```
displayTilt = displayTilt + (targetTilt - displayTilt) blend
```

15.4 Tilt from Burn Vector or Grid Fins

```
tiltX = clamp(-burnz * 0.7, -25, 25) when burn exists, otherwise clamp(gridfinPitch * 0.18, -18, 18)
```

```
tiltZ = clamp(burnx * 0.7, -25, 25) when burn exists, otherwise clamp(-gridfinYaw * 0.18, -18, 18)
```

16. Camera and Reference-View Math

The orbit camera uses standard spherical-coordinate placement with a reference-body auto-orientation mode.

16.1 Reference-Body Framing

```
distToRef = sqrt(dx2 + dy2 + dz2)
```

```
desiredOrbitDistance = max(1.5 distToRef userZoom, 50)
```

```
orbitDistance = orbitDistance + 0.05 (desiredOrbitDistance - orbitDistance)
```

16.2 Auto-Yaw and Pitch

```
targetYaw = atan2(dx, dz)
```

```
orbitYaw = orbitYaw + 0.08 wrapPi(targetYaw - orbitYaw)
```

```
orbitPitch = orbitPitch + 0.08 (0.35 - orbitPitch)
```

16.3 Camera Position from Spherical Coordinates

```
cp = cos(pitch), sp = sin(pitch), cy = cos(yaw), sy = sin(yaw)
```

```
camx = targetx + d cp sy
camy = targety + d sp
camz = targetz + d cp cy
```

This is a conventional spherical-orbit camera, but with auto-yaw chosen to keep the current reference body in a visually readable behind-the-focus spiral composition.

17. What the Space Stack Is, Mathematically

The cleanest technical summary is this:

- The planetary background is a genuine Newtonian N-body simulation in scaled heliocentric coordinates.
- The initial Solar System is built from classical orbital elements via Kepler solving and state-vector rotation.
- The asteroid cloud uses intentionally simplified mass and orbit synthesis to stay visually cheap.
- The Falcon ascent is a local explicit-Euler rigid-body proxy with thrust, constant gravity, quadratic drag, and propellant depletion.
- The interplanetary transfer is an analytic Hohmann-like parametric arc driven by approximate future target position, not by a Lambert solve and not by integrated thrust.
- The approach and landing code implements a bounded PD controller in relative coordinates with explicit gravity compensation.
- The telemetry and camera layers derive heading, pitch, longitude, latitude, delta-v, glow, scale, and orbit framing through straightforward but carefully tuned geometry.

> The most important mathematical caveat is that the Falcon cruise is currently a hybrid of planning math and display math. The transfer planner computes classical orbital quantities, but the runtime cruise state is imposed by parametric interpolation instead of being numerically propagated by rocket thrust or by a general two-point orbital boundary solver.

18. Symbol Index

Symbol	Meaning in This Codebase
a	Semi-major axis of the transfer ellipse or the current orbital element
b	Semi-minor axis used for Ramanujan ellipse-perimeter approximation
e	Eccentricity
E	Eccentric anomaly
M	Mean anomaly
nu	True anomaly or normalized transfer anomaly pi p
mu	Standard gravitational parameter
Omega	Longitude of ascending node
omega	Argument of perihelion
varpi	Longitude of perihelion
r	Radius from the central body
p	Transfer progress in [0, 1] or semi-latus rectum where context requires
Kp, Kd	Proportional and derivative gains for the approach/landing controller
TOF	Transfer time of flight

Respectfully submitted,

Lieutenant Lopez